Project Report

A Gate-Camera Smart Parking System Using Deep-Learning Number Plate Recognition and Occupancy Forecasting

Rishi Heda[1]

[1]*[Department], [University], [City], India*

Correspondence should be addressed to Rishi Heda; rishiheda14@gmail.com

Finding a free parking space wastes time and fuel, and most car parks still rely on a guard at the gate or on a sensor under every space. This report presents a smart parking system that needs only one low-cost camera at each gate. An ESP32-CAM at the entry and the exit streams video to a computer that detects vehicles with YOLO11, finds the number plate with a second detector and reads it with a plate-specific convolutional OCR model. A plate must be read twice before it counts, so single-frame misreads never open the barrier. The system then checks the plate against a vehicle registry, turns away stolen, blocked or unregistered vehicles, keeps a stable space number for every car inside, opens the barrier through a relay and sends alerts. Occupancy is forecast one hour ahead with a seasonal baseline or an LSTM network, and an operator dashboard with a full-screen presentation view shows the state of the lot in real time. On 95 plate detections from six test videos, the plate OCR read 78.7% of the readable plates exactly, against 9.3% for EasyOCR, at about 25 ms per plate on a laptop CPU, and the consensus and format rules prevented every misread from becoming a false entry. Both gates processed real footage simultaneously in real time. On a held-out week of synthetic occupancy data the LSTM forecast was off by 1.79 spaces on average, compared with 2.45 for the seasonal baseline. All software is open, runs on one computer and needs no internet connection.

Keywords: smart parking, automatic number plate recognition, YOLO11, ESP32-CAM, LSTM forecasting, edge computing

1. Introduction

Parking is one of the everyday problems of growing cities. Drivers circle looking for a free space, which wastes fuel, adds to congestion and causes frustration, and car parks that are run by hand need staff at the gate to check vehicles and count free spaces. Smart parking systems aim to automate this: they detect which vehicles are inside, tell drivers whether space is available and control access without a human in the loop [1].

Most published systems take one of two approaches. Sensor-based systems put an infrared, ultrasonic or magnetic sensor at every space and connect them through a microcontroller and Wi-Fi [2, 4, 5]. They are accurate per space but expensive to install and maintain, because the number of sensors grows with the size of the lot. Camera-based systems watch the whole lot from above and classify each space as free or taken [3, 6]. They need one camera with a clear view of every space, which is hard to achieve in covered or irregular car parks, and they do not know *which* vehicle is parked.

This work takes a third approach that suits most real car parks: every vehicle passes through an entry gate and an exit gate, so a camera at each gate can count vehicles and identify them by their number plate. Occupancy is simply the set of plates that have entered and not yet left. Because each car is identified, the same cameras also give access control (registered, blocked or stolen vehicles), a searchable history of every visit, and data for forecasting how full the lot will be.

The contributions of this work are as follows:

(1) A complete gate-camera parking system: ESP32-CAM firmware with a barrier relay, a real-time recognition pipeline, access control, stable space assignment, storage, alerts and an operator dashboard, all running on one computer without internet access.

(2) A recognition pipeline that combines YOLO11 vehicle detection, a dedicated plate detector and a plate-specific OCR model, with a two-read consensus rule, plate-format validation and fuzzy matching against known plates to suppress misreads.

(3) A real-time scheduling scheme for several gates on a CPU: video is analysed at real speed by skipping frames when inference falls behind, and a first-in-first-out lock shares the models fairly between gates.

(4) One-hour occupancy forecasting with a seasonal baseline and an LSTM network, and a synthetic occupancy generator for training before real history exists.

(5) An evaluation on real traffic footage: plate-reading accuracy against hand labels, processing time per stage, an end-to-end run of both gates, and forecasting error on a held-out week.

The remainder of this report is organized as follows. Section 2 reviews related work, Section 3 describes the proposed system, Section 4 presents the results and

discussion, and Section 5 concludes the report.

2. Related Work

Biyik et al. [1] reviewed smart parking systems from a technical point of view, covering the sensors, communication and software architectures in use, and argued that Internet of Things (IoT) technologies will play a central role. Patil et al. [2] detected vehicles entering and leaving with infrared sensors connected to an Arduino Uno and an ESP8266 Wi-Fi module, and guided drivers to empty spaces. Nadipalli et al. [4] also used infrared sensors with a microcontroller and a Wi-Fi modem, and updated a cloud server every 10 seconds, so the system stopped working whenever the network or the microcontroller failed. Subramanian and Subashini [5] proposed a ticket-based system with sensors in every lane and SMS notifications through a mobile application.

Camera-based approaches avoid per-space sensors. Bassam and Samann [3] labelled every space with a character and used optical character recognition (OCR) on an overhead image to find which characters were visible, reporting more than 90% accuracy on eight test images. Alsheikhy et al. [6] used a single overhead camera with image preprocessing and the AlexNet convolutional neural network [8] to count parked vehicles and direct drivers to lanes with free spaces, reporting more than 97% accuracy in MATLAB simulations. Alharbi et al. [7] used OCR to read number plates for a web-based reservation system. These systems either need a view of every space or identify spaces rather than vehicles.

Automatic number plate recognition (ANPR) has improved greatly with deep learning. Single-stage detectors such as YOLO [9] and its successors, including YOLO11 [10], detect vehicles in real time, and transformer-based detectors such as RT-DETR [11] trade speed for accuracy. Laroca et al. [12] built a real-time ANPR system from a cascade of YOLO detectors, and Silva and Jung [13] addressed plates seen at strong angles. For reading the characters, general scene-text engines such as EasyOCR [15], which uses the CRAFT text detector [16], work on any text, while plate-specific models such as fast-plate-ocr [14] are trained only on plate crops.

For forecasting, simple seasonal methods are strong baselines for data with daily and weekly cycles [18], and recurrent networks such as the long short-term memory (LSTM) [17] can learn the pattern and its interaction with the current level. This work combines these elements in one system that runs at the gates of an ordinary car park.

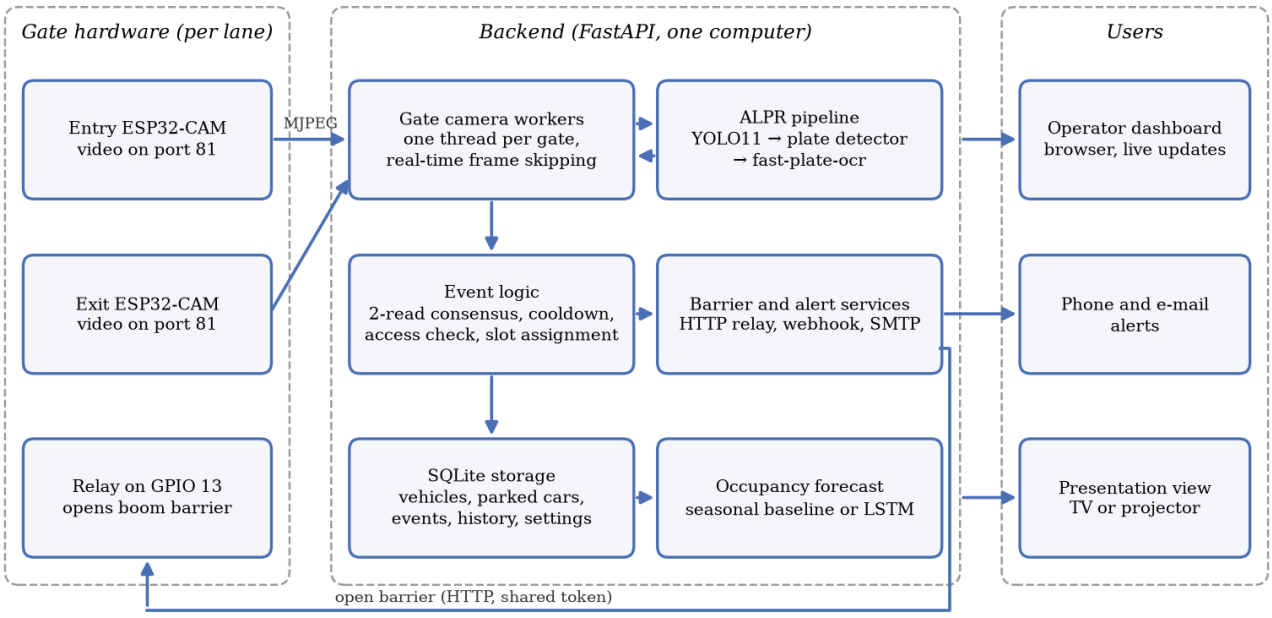


FIGURE 1: System architecture. Each gate has an ESP32-CAM and a barrier relay; one computer runs recognition, event logic, storage, forecasting and the dashboard.

3. The Proposed System

Figure 1 shows the architecture. The system has three parts: the gate hardware, a backend that runs on one computer, and the browser-based interfaces used by operators. Figure 2 shows how each analysed frame becomes a parking event, and Algorithm 1 gives the same logic as pseudocode.

3.1. Gate Hardware. Each lane uses an AI-Thinker ESP32-CAM board. The firmware joins the local Wi-Fi network and runs two HTTP servers: port 81 streams Motion JPEG video for the backend, and port 80 answers `/capture` (one JPEG), `/status` (gate name, uptime, signal strength) and `/open`. The stream has its own port because a stream request never ends and would otherwise block barrier commands. `/open` requires a shared token and pulses a 5 V relay on GPIO 13 whose contacts are wired to the barrier controller's open input; the relay is released from the main loop, so the call returns immediately. If Wi-Fi drops, the backend reconnects to the stream with exponential backoff.

3.2. Recognition Pipeline. For each analysed frame, YOLO11n [10], trained on the COCO data set, detects cars, motorcycles, buses and trucks. A second YOLO model trained for number plates finds the plate boxes, and each crop is read by fast-plate-ocr [14], a compact convolutional model

exported to ONNX that returns the characters with a confidence for each. EasyOCR [15] is kept as a fallback. The text is normalised to capital letters and digits, and reads with a mean character confidence below 0.4 are dropped. A read that is close to a registered or parked plate (string similarity of at least 0.82) is snapped to that plate; otherwise it must match the Indian format `^[A-Z]{2}\d{1,2}[A-Z]{0,3}\d{4}$`, for example MH12AB1234. A permissive mode accepts 4 to 10 letters and digits for foreign test footage.

3.3. From Reads to Events. A single frame can be misread, and a car waiting at the barrier is seen many times. Two rules handle this. First, a camera read only counts once the same plate has been read twice within 10 seconds at that gate. Second, once an event is recorded for a plate at a gate, further reads of that plate at that gate are ignored for a cooldown period (15 s by default, adjustable in the dashboard); for camera reads this includes plates that differ by one character (similarity of at least 0.85), so a one-character misread of the car just seen is not counted as a second car.

At the entry gate the plate is checked against the registry. Stolen, blocked and unregistered vehicles are turned away with the barrier closed and an alert is raised. Registered vehicles are admitted if a space is free: the car is given the lowest free space number, which it keeps until it leaves, and the barrier opens. If the lot is full, the car is turned away. At the exit gate the car's space is freed and the barrier opens; an exit by a car that was never seen entering is recorded as a problem so the operator can check the entry camera. Operators can also record an entry or exit by hand when a camera misses a car.

3.4. Real-Time Processing. Each gate runs in its own thread. A naive loop that analyses every frame falls behind as soon as inference is slower than the video, so a 30-second clip could take many minutes and a live stream would lag ever further. Instead, each worker keeps a real-time clock: for video files it skips frames with the cheap `grab()` call until it reaches the frame that matches the elapsed time, and for live streams it keeps a one-frame buffer. Analysis then runs at a configurable rate (two checks per second by default). Inference is serialised by a first-in-first-out lock, so the entry and exit gates take turns fairly instead of one gate monopolising the models, and the models are warmed up when they are loaded, which removes an 8-second delay from the first car.

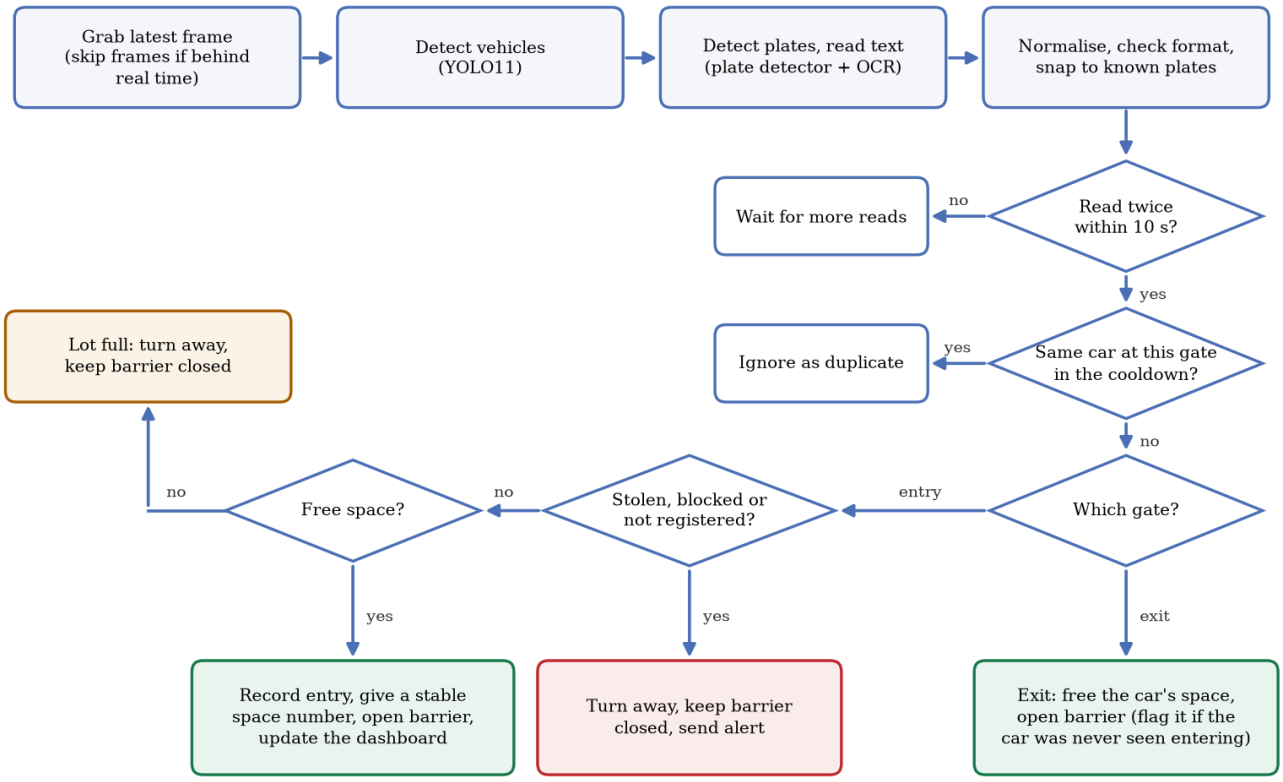


FIGURE 2: Flowchart of the proposed approach, from an analysed frame to a parking event.

3.5. Occupancy Forecasting. Occupancy snapshots are stored every five minutes and after every event. The default forecaster is a seasonal baseline: the history is resampled onto a 15-minute grid, and the forecast for horizon h blends the current level L (an exponentially weighted mean of the last six snapshots) with the mean S of past values in the same weekday and 15-minute slot:

$$F(t+h) = (1 - w)\,L + w\,S, \quad w = (1 - e^{-h/60}) \cdot \min(1, n/4),$$

where F is the forecast and n is the number of matching past samples, so the seasonal pattern is trusted more for longer horizons and as history builds up. The optional LSTM [17] takes the last 16 resampled steps (four hours), each described by five features: occupancy as a fraction of capacity and the sine and cosine of the time of day and of the day of the week. A single LSTM layer with 32 hidden units feeds a linear output that predicts the fraction of capacity four steps (one hour) ahead. It is trained with the Adam optimiser [25] (learning rate 0.01) on mean squared error in PyTorch [19].

Because it predicts a fraction of capacity, weights trained on one lot size can be used for a lot of a different size.

A new car park has no history, so the system includes a synthetic occupancy generator. On weekdays, the occupied fraction follows a logistic rise around 08:15 and fall around 17:45 with a lunchtime dip; Saturdays have a broad afternoon peak and Sundays are quiet. Each day draws its own busyness and arrival and departure times, about one weekday in 25 behaves like a holiday, and first-order autoregressive noise imitates the bursty arrivals of real traffic. The demonstration mode seeds two weeks of this data, and the LSTM can be pre-trained on it until real history is available.

3.6. Storage, Alerts and Security. All state is kept in SQLite, which is built into Python, so the system needs no database server: registered vehicles, cars currently inside with their space numbers, every gate event, occupancy history and dashboard settings live in one file and survive restarts. Events for stolen or blocked vehicles and for a full lot can be sent to a webhook (for example Slack, Discord or ntfy) and by e-mail. When a dashboard password is set, every page, API call, video stream and live update requires sign-in, using an HMAC-signed session cookie; scripts can use a bearer token instead, and failed sign-ins are slowed down.

3.7. Operator Dashboard. The dashboard is a single web page served by the backend and updated live over a WebSocket. It has five pages, each with one job: *Overview* (free spaces, a map of the lot, key figures, the last 24 hours and the latest events), *Gates* (the two camera views, camera controls, barrier buttons and manual entry), *Activity* (searchable history, CSV export and a seven-day report), *Vehicles* (the registry) and *Settings*. A presentation mode shows the overview full-screen with larger text for a projector or a screen at the entrance. Colour is used only for meaning (green for free and allowed, amber for filling up, red for full, stolen or blocked), and all text uses plain words.

The advantages of the proposed system are as follows:

(i) Low cost: one camera board and one relay per gate, regardless of the number of spaces.

(ii) It identifies vehicles, so it provides access control, alerts and a full history, not only counts.

(iii) It runs on one computer without internet access, so it keeps working during network outages.

(iv) It is easy to operate: plain-language dashboard, manual override and a presentation view.

(v) It is easy to extend: billing, reservations and driver notifications can be added using the stored events.

The system also has limitations. Occupancy depends on every vehicle being read at both gates, so a missed read leaves a wrong count until an operator corrects it, and it does not detect cars parked outside the marked spaces. A dead camera stops automatic counting at that gate; the dashboard shows the camera state, and manual entry keeps the lot running until it is fixed.

Input: video from the entry and exit cameras; vehicle registry; capacity C.
Output: parking events, occupancy, space numbers, barrier commands and alerts.
1. **For each gate, repeatedly do:**
2. Skip to the frame that matches the elapsed real time.
3. Detect vehicles; detect plates; read each plate crop with the OCR model.
4. Normalise each read; drop it if its confidence is below 0.4.
5. Snap it to a known plate if similarity ≥ 0.82, else require a valid plate format.
6. **If** the plate was not read twice within 10 s **then** continue.
7. **If** the same (or one-character-different) plate had an event at this gate within the cooldown **then** continue.
8. **If** gate = entry **then**
9. **If** the plate is stolen, blocked or unregistered **then** turn away, keep the barrier closed, send an alert.
10. **Else if** occupancy = C **then** turn away (lot full).
11. **Else** record the entry, assign the lowest free space, open the barrier.
12. **Else** (exit gate): free the car's space and open the barrier; flag the exit if no entry is on record.
13. Save the event, bump the state version so dashboards update, and record an occupancy snapshot.
14. **End**

ALGORITHM 1: Gate processing and access control.

TABLE 1: Experimental setup.

Component	Details
Computer	Laptop, 12th Gen Intel Core i7-1255U (10 cores, 12 threads), 15.7 GB RAM, no GPU used
Operating system	Windows 11 Home, Python 3.11.9
Detection	Ultralytics 8.4.138: YOLO11n (vehicles), YOLO plate detector; PyTorch 2.14 (CPU)
OCR	fast-plate-ocr 1.1.0 (cct-s-v2-global-model, ONNX Runtime 1.31); EasyOCR 1.7.2 for comparison
Backend	FastAPI 0.143, SQLite 3.45, OpenCV 5.0
Gate hardware	AI-Thinker ESP32-CAM, 5 V opto-isolated relay module (GPIO 13)
Test footage	Two supplied gate clips (1920×1080), one short car clip (420×380), Intel Open Edge Platform parking video [22] (1920×1080), two Indian traffic clips from Pexels [23, 24] (3840×2160)

4. Results and Discussion

The system was evaluated on the setup in Table 1. All processing ran on the laptop CPU. Because the gate hardware had not yet been installed at a car park, the cameras were fed with recorded traffic videos, which the backend treats exactly like a live stream. Four scenarios demonstrate the behaviour of the system, followed by measurements of plate-reading accuracy, processing time and forecasting error.

4.1. Scenario 1: Lot Full. The lot was set to eight spaces with eight cars inside, and a registered car arrived at the entry gate. Figure 3 shows the result: the sign shows 0 free spaces in red with the status *Lot full*, every space on the map is taken, and the latest event reads *Turned away: the lot was full*. The barrier stayed closed. While reproducing this scenario we found and fixed a defect: after reducing the capacity, a car in a space beyond the new limit disappeared from the map. Such cars are now moved into free spaces and the move is stored, so later exit events name the right space; an automated test covers this case.

4.2. Scenario 2: Normal Operation. Figure 4 shows the overview of a twelve-space lot during normal operation. Registered cars that arrive are admitted and keep the space number they were given; cars that leave free their space, and the barrier opens in both cases. The figures row shows the cars inside, the cars that came in today, the vehicles turned away and the forecast for the next hour with its likely range. The chart shows the last 24 hours, with the forecast as a point and range one hour ahead.

4.3. Scenario 3: Stolen and Blocked Vehicles. A vehicle reported stolen (DL8CAF5031) and a vehicle blocked from the lot (MH14XY9087) arrived at the entry gate. Both were turned away with the barrier closed, and an alert was raised for each. Figure 5 shows the Activity page, where these events appear in red with the reason (*Reported stolen*, *Blocked from parking here*) and can be isolated with the *Problems* filter or exported as CSV.

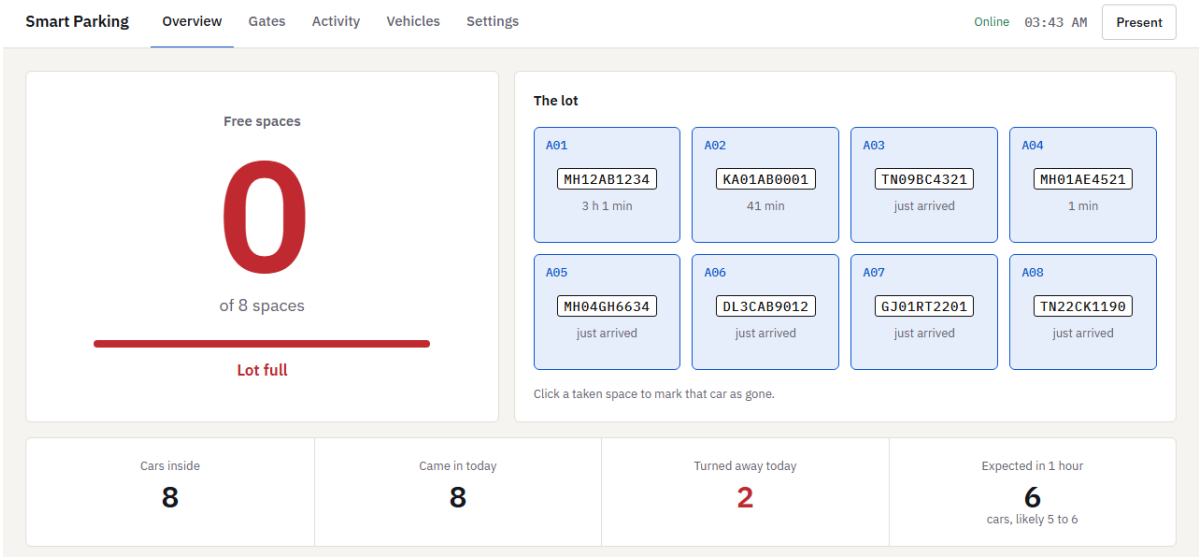


FIGURE 3: Scenario 1: the lot is full and an arriving car is turned away.

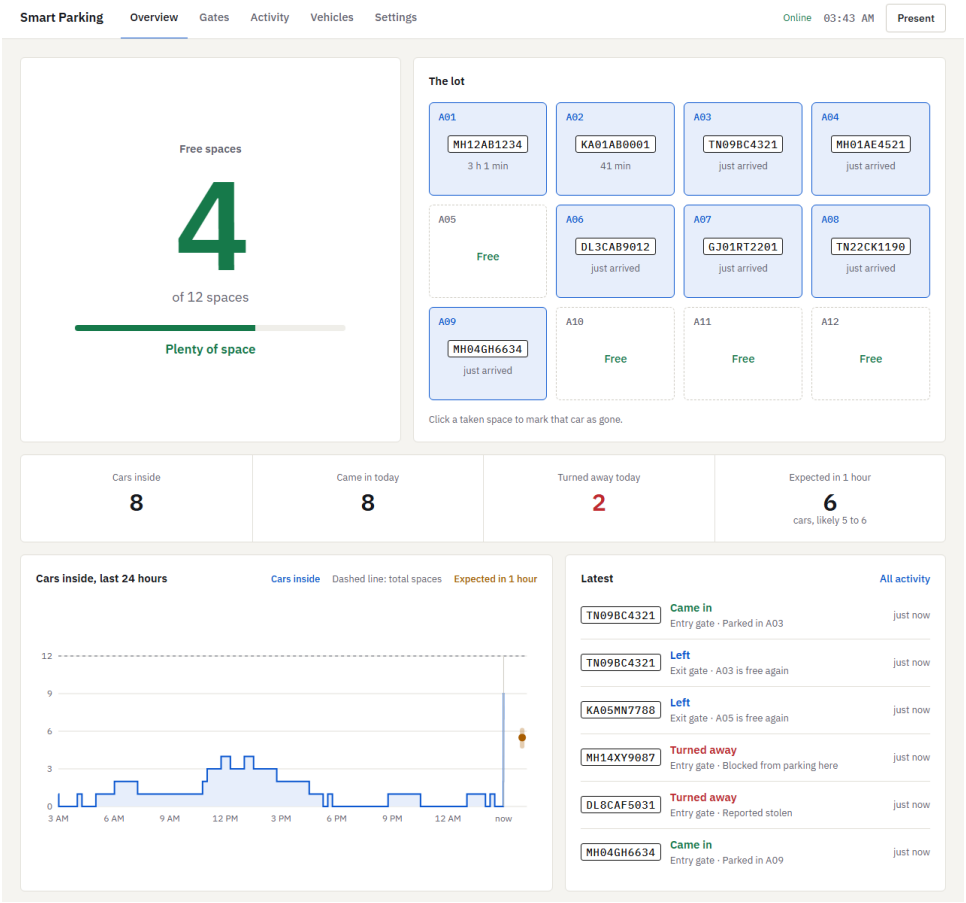


FIGURE 4: Scenario 2: overview page during normal operation of a twelve-space lot.

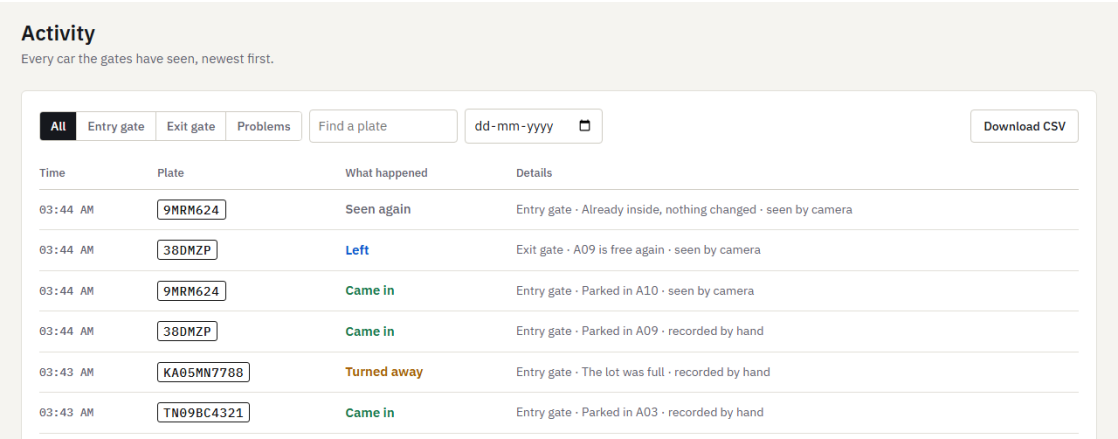


FIGURE 5: Scenario 3 and 4: Activity page with turned-away stolen and blocked vehicles, a lot-full rejection, and the camera events from Scenario 4.

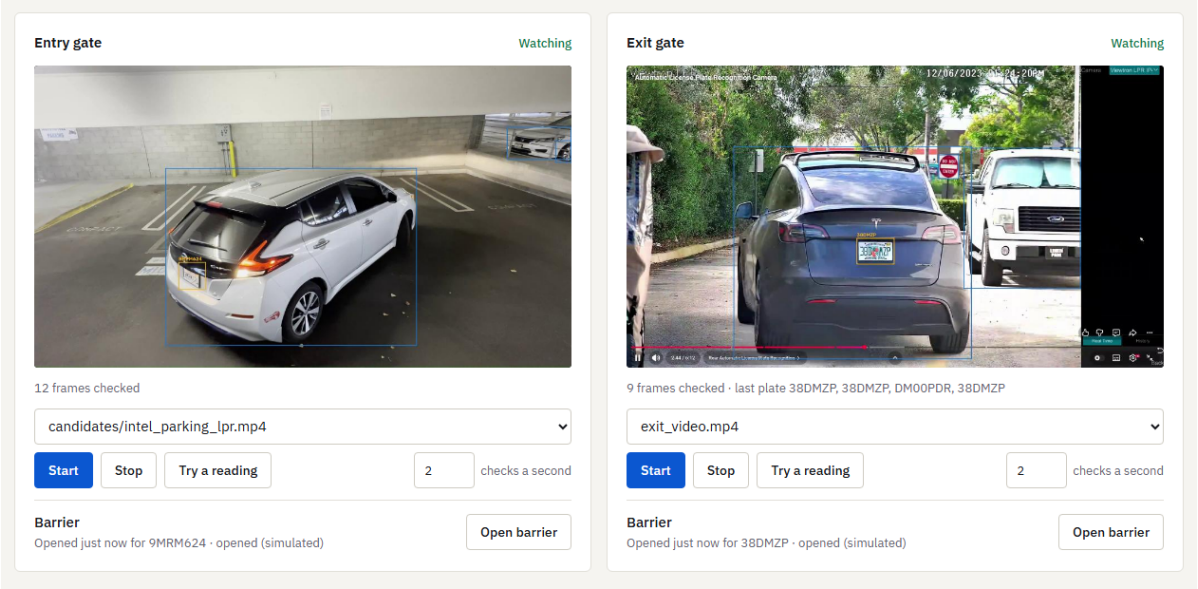


FIGURE 6: Scenario 4: both gates analysing real footage at the same time. Vehicle boxes are blue and plate boxes yellow; the barrier line shows the last opening.

4.4. Scenario 4: Real Footage at Both Gates. The detector was switched from demonstration mode to YOLO11, which loaded and warmed up the models in 6 to 12 s. The entry camera was given the Intel parking video [22] (30.8 s, a car with plate 9MRM624 entering a garage) and the exit camera a 11.3 s clip of a car with plate 38DMZP, both at two checks per second, at the same time. Both plates were registered and 38DMZP was recorded as inside beforehand. Figure 6 shows the Gates page during the run.

Both videos were finished 34.4 s after they were started, so the two gates kept up with real time while sharing one CPU: the entry worker analysed 42 frames and skipped 704, and the exit worker analysed 11 and skipped 329. The system recorded exactly two state changes: 9MRM624 entered and was given a space, and 38DMZP exited and freed its space, each opening the corresponding barrier. The entry car stayed in view for longer than the 15 s cooldown, so it was seen once more and recorded as *already inside*, which changes nothing. No false entry or exit was recorded. The same run repeated on a separate occasion gave identical events.

4.5. Plate Recognition Accuracy. To measure accuracy, frames were sampled every 0.5 s from six videos (up to 60 frames each, 189 frames in total) and passed through the vehicle and plate detectors. The plate detector returned 95 boxes. Each crop was labelled by hand: 75 were readable plates (8 distinct plates), 15 were plates too blurred, angled or cut off for a person to read, and 5 were not plates (for example a motorcycle badge and a sticker). Each readable crop was read by fast-plate-ocr and by EasyOCR. Table 2 gives the results and Figure 8 shows examples.

fast-plate-ocr read 78.7% of the readable crops exactly and 86.7% to within one character, with a character error rate of 6.5%. EasyOCR read only 9.3% exactly; it read the clean, large Indian plate MP04RB0693 well (7 of 8) but failed on the others, typically merging decorative text around the characters into the result (for example “J81GMZP” for 38DMZP). The plate model was also 6 to 30 times faster (Table 3). Its errors were concentrated on three kinds of plates: heavily compressed, colour-tinted footage (the TN plates), two-line plates with thin characters (MP04ZU2506) and a red-bordered plate (MP34CA5252). The general model used here is not trained specifically on Indian plates, so fine-tuning on Indian data is the most direct improvement.

What matters for the parking system is whether a misread becomes a false event. Of the 16 incorrect reads that passed the confidence threshold, only 3 matched the Indian plate format, and none of them was repeated within the consensus window, so **no false entry or exit would have been recorded**. Four of the eight distinct plates were confirmed correctly by two matching reads; for the other four the system would record nothing, and an operator would add the car by hand. With the permissive foreign-plate format, two wrong plates would have been confirmed, which shows the value of the format rule.



FIGURE 7: Pipeline output on real footage: vehicle boxes (blue) and plate boxes (yellow) with the plate text and confidence. Right: the badge of an auto-rickshaw is detected as a plate, but produces no valid read.



FIGURE 8: Examples of plate crops with the hand label and the fast-plate-ocr read. Top row: correct reads. Bottom row: failure cases and a non-plate detection.

TABLE 2: Plate recognition accuracy on hand-labelled crops.

Measure	fast-plate-ocr	EasyOCR
Readable plate crops (8 distinct plates)	75	75
Exact reads	59 (78.7%)	7 (9.3%)
Reads within one character	86.7%	13.3%
Mean character error rate	6.5%	67.7%
Exact reads by plate: 38DMZP / 9MRM624	22 of 23 / 30 of 31	0 of 23 / 0 of 31
Exact reads by plate: MP04RB0693 / FL020EH	5 of 8 / 2 of 3	7 of 8 / 0 of 3
Exact reads by plate: TN plates / MP04ZU2506 / MP34CA5252	0 of 6 / 0 of 2 / 0 of 2	0 of 6 / 0 of 2 / 0 of 2
Wrong reads passing format check (Indian mode)	3 of 16	–
False events after the two-read consensus	0	–

TABLE 3: Mean processing time per stage in milliseconds on the laptop CPU (after warm-up).

Video (resolution)	Frames	Vehicle detection	Plate detection	fast-plate-ocr (per plate)	EasyOCR (per plate)
Entry clip (1920×1080)	55	53.6	47.6	29.1	368.4
Exit clip (1920×1080)	23	63.9	48.4	25.1	395.4
Car clip (420×380)	8	166.0	115.6	20.0	147.2
Intel parking (1920×1080)	60	85.7	60.2	23.5	323.9
India highway (3840×2160)	29	81.7	64.6	32.8	1019.2
India street (3840×2160)	14	71.9	64.9	23.2	611.4

4.6. Processing Speed. Table 3 shows the mean time per stage. Vehicle detection took 54 to 86 ms per frame and plate detection 48 to 65 ms on full-HD and 4K frames; YOLO resizes every frame to the same input size, so 4K frames cost little more than full HD. The short low-resolution clip had only eight frames with many vehicles and shows higher, less reliable means. Reading a plate took 20 to 33 ms with fast-plate-ocr against 147 to 1019 ms with EasyOCR. A frame with one plate therefore takes about 0.13 to 0.18 s, so one CPU can analyse three to four frames per second per gate for two gates, which is more than the two checks per second needed for cars moving slowly through a barrier.

4.7. Occupancy Forecasting. Fifty-six days of synthetic occupancy were generated for a 50-space lot (Figure 9 shows two weeks). The models were trained on the first 49 days (4,685 training windows for the LSTM, 300 epochs, about 25 s on the CPU) and tested on the last 7 days, predicting from the six hours before each 15-minute step. Table 4 gives the mean absolute error. One hour ahead, the LSTM was off by 1.79 spaces on average, the seasonal baseline by 2.45 and a persistence forecast (no change) by 4.24. Figure 10 shows that both models follow the daily pattern, and the LSTM reacts faster to the morning rise and evening fall. Persistence is best at 15 minutes, because occupancy changes little in such a short time; the LSTM is trained only for the one-hour horizon used by the dashboard. These results are on synthetic data, so they show that the models work, not how accurate they will be at a particular car park; the models must be retrained on real history once it exists.

4.8. Software Verification. The backend has 55 automated tests that run in about 17 seconds. They cover the summary and events API, stable space assignment, the lot-full and capacity rules, stolen and blocked vehicles, the consensus and cooldown rules including one-character misreads, persistence of data and settings across restarts, sign-in, the barrier relay against a local HTTP server, webhook alerts, the camera worker on a generated video (including frame skipping when analysis is slow), the forecasting models and the synthetic data generator.

4.9. Comparison. Table 5 compares the proposed system with related work in terms of method, equipment, accuracy and cost. The figures for other systems are those reported by their authors and were obtained on different data, so they are not directly comparable. Unlike the sensor-based systems [2, 4, 5], the proposed system needs no equipment per space; unlike the overhead-camera systems [3, 6], it does not need a view of every space, and it identifies each vehicle, which enables access control and alerts. Its cost is two camera boards, two relays and one computer, and it needs no internet connection.

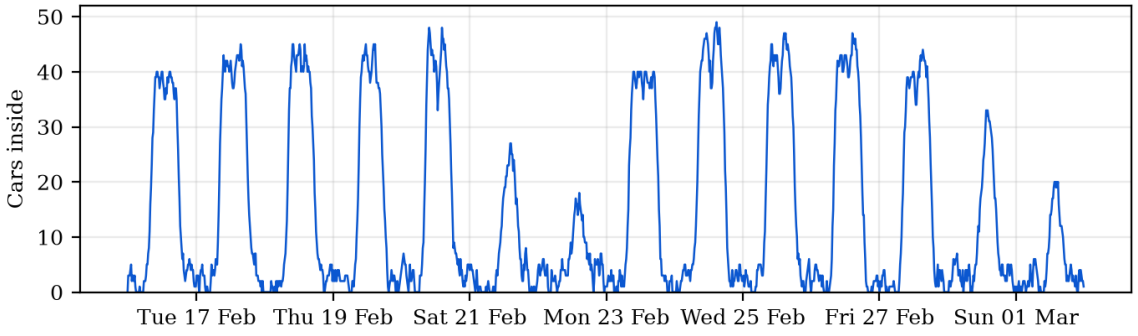


FIGURE 9: Two weeks of synthetic occupancy for a 50-space lot used for training.

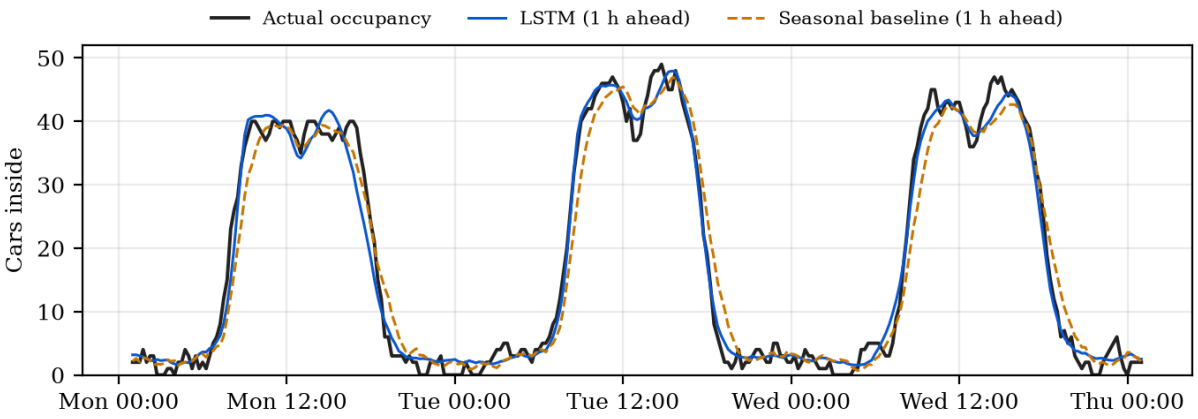


FIGURE 10: Actual occupancy and one-hour-ahead forecasts on three held-out days.

TABLE 4: Mean absolute forecasting error in spaces on the held-out week (50-space lot). The LSTM is trained for the 1-hour horizon only.

Forecast horizon	LSTM	Seasonal baseline	Persistence (no change)
15 minutes	–	1.90	1.46
30 minutes	–	2.26	2.45
1 hour	1.79	2.45	4.24

TABLE 5: Comparative evaluation with related work (figures as reported by each work).

Work	Methodology	Utilized equipment	Accuracy	Cost
Patil et al. [2]	IR sensing at entry and exit	Arduino, IR sensors, Wi-Fi module	Not mentioned	Expensive
Bassam and Samann [3]	Optical character recognition of labelled spaces	Camera, web and e-mail server, database, internet	>90%	Expensive
Nadipalli et al. [4]	IR sensing with cloud server	IR sensors, DC motor, Wi-Fi modem, microcontroller	Not mentioned	Expensive
Subramanian and Subashini [5]	IoT sensors and mobile application	Database, internet, phone, wireless sensors, server	Not mentioned	Expensive
Alsheikhy et al. [6]	CNN (AlexNet) on an overhead image	Single overhead camera	>97% (simulation)	Cheap
Proposed system	YOLO11 + plate detector + plate OCR at the gates; consensus rules; LSTM forecast	One ESP32-CAM and relay per gate, one computer	78.7% exact plate reads per frame; no false events (Section 4.5)	Cheap

5. Conclusion

This report presented a smart parking system that identifies vehicles at the entry and exit gates with one low-cost camera per gate. A pipeline of YOLO11 vehicle detection, a dedicated plate detector and a plate-specific OCR model, combined with two-read consensus, plate-format validation and fuzzy matching, read 78.7% of readable plate crops exactly at about 25 ms per plate, and no misread became a false event. Real-time scheduling let two gates process real footage simultaneously on a laptop CPU, and the system handled full lots, stolen and blocked vehicles, stable space numbers, barrier control and alerts. One-hour forecasts with an LSTM were more accurate than a seasonal baseline on synthetic data, and a clear operator dashboard with a presentation view makes the state of the lot visible at a glance.

Future work will flash and install the ESP32-CAM boards at a real car park and measure accuracy on its own cameras, fine-tune the plate OCR on Indian plates (especially two-line and tinted plates), retrain the forecaster on real occupancy history, and add features enabled by the stored events, such as billing, reservations and notifications to drivers.

Data Availability

The Intel parking video is available from the Open Edge Platform repository [22], and the Indian traffic videos from Pexels [23, 24] under the Pexels licence. The hand labels, benchmark results and forecasting results are included with the project in `docs/paper_figures`.

Conflicts of Interest

The author declares that there is no conflict of interest regarding this work.

Acknowledgments

[Add acknowledgments here.]

References

[1] C. Biyik, Z. Allam, G. Pieri et al., “Smart parking systems: reviewing the literature, architecture and ways forward,” *Smart Cities*, vol. 4, no. 2, pp. 623–642, 2021.

[2] V. Patil, A. Randive, and D. Gaikwad, “Smart car parking system,” *Open Access International Journal of Science and Engineering*, vol. 6, pp. 45–47, 2021.

[3] R. Bassam and F. Samann, “Smart parking system based on improved OCR model,” *IOP Conference Series: Materials Science and Engineering*, vol. 978, no. 1, article 012007, 2020.

[4] L. S. P. S. Nadipalli, K. P. Kishore, B. M. Reddy, K. V. Kishna, and D. R. S. Aravind, “A cloud based smart parking system using IoT technology,” *International Journal of Recent Technology and Engineering*, vol. 8, no. 5, pp. 96–100, 2020.

[5] K. Subramanian and T. S. Subashini, “Ticket based smart parking system using mobile application in vehicular network,” *European Journal of Molecular and Clinical Medicine*, vol. 7, no. 3, pp. 685–695, 2020.

[6] A. A. Alsheikhy, T. Shawly, Y. F. Said, and H. Lahza, “An intelligent smart parking system using convolutional neural network,” *Journal of Sensors*, vol. 2022, Article ID 7571716, 2022.

[7] A. Alharbi, G. Halikias, M. Yamin, and A. A. Abi Sen, “Web-based framework for smart parking system,” *International Journal of Information Technology*, vol. 13, no. 4, pp. 1495–1502, 2021.

[8] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” in *Advances in Neural Information Processing Systems*, vol. 25, 2012.

[9] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: unified, real-time object detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 779–788, 2016.

[10] G. Jocher and J. Qiu, “Ultralytics YOLO11,” 2024, https://github.com/ultralytics/ultralytics.

[11] Y. Zhao, W. Lv, S. Xu et al., “DETRs beat YOLOs on real-time object detection,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.

[12] R. Laroca, E. Severo, L. A. Zanlorensi et al., “A robust real-time automatic license plate recognition based on the YOLO detector,” in *Proceedings of the International Joint Conference on Neural Networks*, 2018.

[13] S. M. Silva and C. R. Jung, “License plate detection and recognition in unconstrained scenarios,” in *Proceedings of the European Conference on Computer Vision*, pp. 580–596, 2018.

[14] “fast-plate-ocr: lightweight license plate OCR,” GitHub repository, https://github.com/ankandrew/fast-plate-ocr.

[15] JaidedAI, “EasyOCR,” GitHub repository, https://github.com/JaidedAI/EasyOCR.

[16] Y. Baek, B. Lee, D. Han, S. Yun, and H. Lee, “Character region awareness for text detection,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 9365–9374, 2019.

[17] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[18] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed., OTexts, Melbourne, 2021.

[19] A. Paszke, S. Gross, F. Massa et al., “PyTorch: an imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[20] S. Ramírez, “FastAPI,” https://fastapi.tiangolo.com.

[21] Espressif Systems, “esp32-camera,” GitHub repository, https://github.com/espressif/esp32-camera.

[22] Intel Open Edge Platform, “edge-ai-resources: ParkingVideo.mp4,” https://github.com/open-edge-platform/edge-ai-resources.

[23] Pexels, “Motorcyclists commuting on Indian highway,” video 34395449, https://www.pexels.com/video/motorcyclists-commuting-on-indian-highway-34395449/.

[24] Pexels, “Motorcyclist riding through urban Indian street,” video 34394431, https://www.pexels.com/video/motorcyclist-riding-through-urban-indian-street-34394431/.

[25] D. P. Kingma and J. Ba, “Adam: a method for stochastic optimization,” in *Proceedings of the International Conference on Learning Representations*, 2015.